

MCU 加密位、熔丝位、USART 接口实验思考报告

杜晓阳 PB23050123

一、实验目的与内容回顾

1.1 实验目的

本次实验旨在掌握 ATMEGA8A 单片机的加密位与熔丝位配置方法，熟悉 USART 接口的工作原理及编程应用，并通过实际操作验证硬件通信功能。通过实验加深对 MCU 底层配置、时钟系统及串口通信机制的理解。

1.2 实验内容

■ ATMEGA8A 的加密位（保护芯片内的程序）熔丝位与加密位配置：学习通过 ProgISP 工具设置 ATMEGA8A 的加密位（Boot Lock Bit 与 Lock Bit）及熔丝位（高字节与低字节），理解其对程序保护与时钟配置的影响。

■ ATMEGA8A 的 USART 接口（通用串口）：USART 接口应用：掌握 USART 模块的寄存器配置、波特率计算及帧格式设置，通过代码实现异步通信，完成 MCU 间双向数据传输与 LCD 显示。

时钟源配置：根据外部晶体或内部 RC 振荡器，按位配置熔丝位以设置工作时钟（如 8MHz、1MHz 等）。

综合实验：结合加密、熔丝位设置与 USART 通信，完成 PC 端与 MCU 的数据交互及功能验证。

二、关键知识点分析与思考

2.1 熔丝位与加密位的配置

2.1.1 加密位的作用与风险

加密位功能：通过设置 Boot Lock Bit（BLB）与 Lock Bit（LB），可防止程序被读取或意外擦除。文档中提及 BLB 默认值为 11111111，需从 1 编程为 0，反向操作需全局擦除芯片 5。

实践思考：若误将加密位设置为锁定状态，可能导致芯片无法通过 ISP 重新编程。因此，配置前需明确需求，并备份当前熔丝位状态。

2.1.2 熔丝位的高字节与低字节配置

高字节（熔丝 H）：控制复位引脚功能（RSTDISBL）、看门狗（WDTON）、SPI 编程使能（SPIEN）等。例如，RSTDISBL=1 时，引脚 1 作为 PC6 使用而非复位 2。

低字节（熔丝 L）：设置 BOD 检测电平（BODLEVEL）、启动时间（SUT）及时钟源选择（CKSEL）。时钟源配置需根据外接晶体或内部 RC 调整 CKSEL[3:0]，如外部 16MHz 晶体对应 101024。

实践思考：熔丝位配置需严格遵循“读出→修改→写入”流程。若直接写入错误值，可能使芯片无法启动。建议先用向导模式生成配置，再手动微调。

2.2 USART 接口的工作原理与配置

2.2.1 波特率计算与误差分析

公式推导：异步普通模式下，波特率 $BAUD = f_{osc} / (UBRR + 1) \times 16$ ，需根据晶振频率 f_{osc} 与目标波特率计算 UBRR 值。例如， $f_{osc} = 1\text{MHz}$ ，目标波特率 4800bps 时， $UBRR = 12$ （误差 0.2%）。

2.2.2 数据帧格式与寄存器操作

帧结构：支持 5~9 位数据、1 或 2 位停止位，可选奇偶校验。数据发送时，最低位先传输，接收后需判断帧错误标志（FE）。

寄存器协作：

UCSRA：标志位（RXC、TXC）指示接收/发送完成，U2X 控制倍速模式。

UCSRB：中断使能（RXCIE、TXCIE）与收发器开关（RXEN、TXEN）。

UCSRC：配置模式（UMSEL）、校验（UPM）、停止位（USBS）及字符位数（UCSZ）。

UDR：共享发送/接收缓冲区，写操作触发发送，读操作获取接收数据 51819。

实践思考：USART 初始化需按顺序配置寄存器。例如，先设置波特率（UBRR），再开启收发器（RXEN、TXEN），最后配置帧格式。若顺序颠倒，可能导致配置失效。

三、实验具体操作

3.1 实验内容 1：设置实验板的工作时钟为 8MHz（如果外接了晶体，也可以设置为其他的时钟，如 11.0592M 等）

问题现象：误将 RSTDISBL 设为 0 后，MCU 无法通过复位引脚重启。

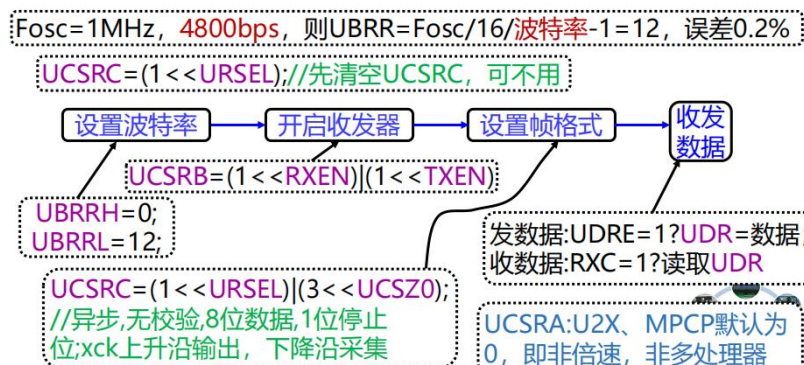
解决方案：通过高电压编程器（如并行编程）擦除芯片并重新配置熔丝位。后续操作前需仔细核对熔丝位功能表 2。

其中用 progisp 设置加密与熔丝位，一定要遵守相应的规则，先点击“读出”正确读出后，再根据使用需求进行正确设置（鼠标左键点击相应位设置为 0 或 1），确认无误后再点击“写入”；意外或不确定的设置会导致 MCU 芯片无法正常使用或须经过特殊处理后才能再用，同时在调整完 8MHz 后要恢复默认设置，以便于后面的实验任然事 1MHz。

时钟源	CKSEL3:0	频率范围 MHz	CK SEL0	SUT 1:0	STPP (CK)	ADR (ms)	推荐用法
外部陶瓷谐振器	CKOPT=1 101x	0.4~0.9	0	00/11	258/1K	4.1	陶瓷,快升
外部晶体谐振器	CKOPT=1 110x	0.9~3.0	0/1	01/00	258/1K	65	陶瓷,慢升
	CKOPT=1 111x	3.0~8.0	0	10	1K	-	陶瓷,BOD
	CKOPT=0 101x	1.0~16.0	1	01	16K	-	晶体,BOD
	CKOPT=0 110x		1	10	16K	4.1	晶体,快升
	CKOPT=0 111x		1	11	16K	65	晶体,慢升
外部低频晶体	x 1001	32.768kHz	x	00/01/10	1K/1K/32K	-	快/慢/稳
外部RC振荡器	x 0101	0.1~0.9	1	00	18	-	BOD开启
	x 0110	0.9~3.0	0	01	18	4.1	快升电源
	x 0111	3.0~8.0	1	10	18	65	慢升电源
	x 1000	8.0~12.0	0	11	6	4.1	快升/BOD
校准内部RC振荡器	x 0001	1.0	1	00	6	-	BOD开启
	x 0010	2.0	0	01	6	4.1	快升电源
	x 0011	4.0	1	10	6	65	慢升电源
	x 0100	8.0	0	11	-	-	保留
外部时钟	x 0000	0.1~16.0	0	-	-	-	与校准内部RC振荡器相同

3.2 实验内容 2：测试自己实验板的 USART 是否工作正常

将自己的 PD1 连接自己的 PD0，UART 自测：通过 TXD 发送可以在 LCD 上显示的字符，再通过 RXD 接收数据并显示在 LCD 的第 2 行。



3.3 实验内容 3：与同学合作，通过 USART 进行双向数据的传递与显示，如可以在对方 LCD 第 2 行或第 1 行显示己方的开关次数等

在这个实验当中首先要将线路连接正确，其中 2 个板子的 RXD 和 TXD 要求交叉连接，其中关键对于收发程序，两者分别写入不同的收发程序。

对于第一个实验代码：是基于 ATmega8A 微控制器的程序，主要功能是利用定时器 T0 统计开关次数，并通过 USART 接口进行数据的发送和接收，同时将接收到的数据显示在 TWI 连接的 LCD 屏幕上。

头文件和宏定义

```
1 #define F_CPU 1000000UL
2 #include <avr/io.h>
3 #include "twi_lcd.h"
4 #include <util/delay.h>
```

- #define F_CPU 1000000UL : 定义系统时钟频率为 1MHz。
- <avr/io.h> : 包含 AVR 微控制器的输入输出寄存器定义。
- "twi_lcd.h" : 自定义头文件，包含 TWI 接口 LCD 屏幕的相关函数声明。
- <util/delay.h> : 包含延时函数。

主循环

```
1 while (1)
2 {
3     if(tp_r!=TCNT0) // 开关次数发生变化
4     {
5         tp_r = TCNT0;
6         // 读取TCNT0, 即新的开关次数
7         while(!(UCSRA & (1<<UDRE)));
8         // 可以发送数据?
9         UDR = tp_r; // 发送数据
10    }
11    if(UCSRA & (1<<RXC)) // 有收到数据?
12    {
13        LCD_Write_Char(1,i,UDR); // 显示
14        i++;
15    }
16    if(i>15)i=0; // 逐次显示, 并回退
17    _delay_ms (1);
18 }
19 }
```

- if(tp_r!=TCNT0) : 检查开关次数是否发生变化，如果变化则将新的开关次数通过 USART 发送出去。
- while(!(UCSRA & (1<<UDRE))); : 等待 USART 数据寄存器为空，即可以发送新数据。
- UDR = tp_r; : 将新的开关次数写入 USART 数据寄存器进行发送。
- if(UCSRA & (1<<RXC)) : 检查 USART 是否接收到新数据，如果接收到则将数据显示在 LCD 屏幕的第 1 行。
- if(i>15)i=0; : 当显示位置超过 LCD 屏幕第 1 行的最大列数时，将显示位置重置为 0。
- _delay_ms (1); : 主循环每次执行后延时 1ms。

对于第 2 块板子，其接受端

PPT 中给出的事例其缺少数组 tmp 的初始化定义，由此对其进行补充完整。

```

1 // ... existing code ...
2 unsigned char i=0,tp_r=0; //临时变量
3 char tmp[4]; // 定义数组
4 memset(tmp, ' ', sizeof(tmp)); // 正确顺序：指针、填充值、大小
5 tmp[3] = '\0'; // 字符串终止符
6 // ... existing code ...

```

- 在 main 函数起始处添加定义： `char tmp[4]`
- 数组大小 4 的原因：十进制数最大 255（3 位数字） + 1 字节缓冲区数组初始化建议保持 3 字节空格+1 字节终止符的格式，便于 LCD 显示三位数字
- 根据后续代码逻辑，该数组用于存储数字转换后的 ASCII 字符（0x30-0x39）和填充空格（0x20）

-memset 参数：代码 `memset(tmp,sizeof(tmp))` 可以对其进行初始化定义

问题分析：MCU1 发送数据后，MCU2 未正确接收，可能因波特率不匹配或接收中断未使能。

解决步骤：

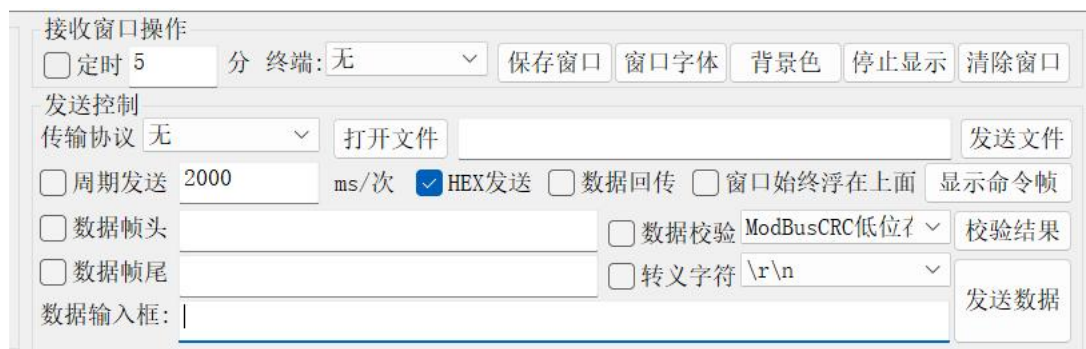
检查双方 UBRR 值是否一致（如 1MHz 下 4800bps 对应 UBRR=12）6。

确认 UCSRB 中 RXEN 已开启，且 UCSRC 帧格式匹配（如 8 位数据、1 停止位）519。

启用接收中断（RXCIE）或轮询 RXC 标志位 520。

3.3 实验内容 4：通过 USART 和 USB-TTL 模块与电脑进行双向数据的传递与显示

首先连接电路后对其插入 USB 接口，打开电脑管理器，查看接口名称，随后安装驱动，打开串行软件接口，重点是要对于串口设置，使其串口号，频率等要符合设备，对于接收端口设置完发送即可。



工作流程：

flowchart TB

初始化 USART-->初始化 LCD

等待接收-->校验指令长度

校验指令长度-->|收到 2 字节|解析操作码

解析操作码-->执行 LCD 操作

执行 LCD 操作-->返回 OK 响应

对于发送内容：往 LCD1602 发送要显示的字符：需输入“5d+字符 ASCII 值”后，点击“发送数据”；往 LCD1602 发送控制命令：需输入“5c+LCD 命令”后，点击“发送数据”。

实际有效指令结构：

1. 5C+XX：LCD 控制指令

- 示例：5C 80 表示设置 DDRAM 地址到第一行起始

2. 5D+XX：字符显示指令

- 示例：5D 41 表示显示字符'A'

可能问题：LCD 显示异常

问题原因：USART 接收数据后未正确解析为 LCD 指令或字符。

调试方法：

检查 LCD 写入函数是否匹配指令格式（如 0x5c 为写命令，0x5d 为写数据）10。

四、实验收获与改进建议

知识收获

底层配置能力：掌握熔丝位对 MCU 功能的核心影响，理解时钟系统与加密机制的底层逻辑。

通信协议实践：通过 USART 编程，熟悉串口通信的参数配置、数据收发流程及错误处理。

软硬件协同：学会结合硬件连接（如晶体选型、跳线帽配置）与软件配置（如熔丝位、USART 初始化）实现功能。

五、总结

本次实验通过配置 ATMEGA8A 的加密位、熔丝位及 USART 接口，深入理解了 MCU 底层机制与通信原理。实践中遇到的问题（如熔丝位误操作、波特率误差）促使我更严谨地对待硬件配置，并学会通过调试工具与代码优化解决问题。未来可进一步探索高级通信协议与低功耗设计，提升嵌入式系统开发能力。